

BÀI THỰC HÀNH

LAB 11

Giám sát dữ liệu chuyển động với MPU6050

Đọc gia tốc và vận tốc góc bằng MPU6050 Device API với EduFrame-
work

S32K144 · MaaZEDU

Mục lục

1	Giới thiệu	2
1.1	Tổng quan bài lab	2
1.2	Mục tiêu	2
2	Kiến thức nền	2
2.1	gia tốc kế và con quay hồi chuyển	2
2.2	Giao tiếp I2C	3
3	Thiết lập phần cứng	3
3.1	Phần cứng sử dụng	3
3.2	Ánh xạ tín hiệu I2C	3
3.3	Kết nối MPU6050	3
4	EduFramework API	4
4.1	Cấu trúc MPU_Data_t	4
4.2	MPU_Begin()	5
4.3	MPU_ReadData()	5
5	Bài thực hành	6
5.1	Yêu cầu	6
5.2	Chương trình	6
5.3	Giải thích chương trình	7
5.4	Kiểm chứng	8
6	Mở rộng	8
6.1	Đọc chọn lọc và kiểm tra trạng thái	8
6.2	Hiệu chuẩn với MPU_Calibrate()	9
6.3	Địa chỉ I2C và chân AD0	9
6.4	Tùy chỉnh dải đo	10
6.5	Advanced API theo context	10
6.6	Bài tập mở rộng	11
7	Tài liệu tham khảo	11

1 Giới thiệu

1.1 Tổng quan bài lab

MPU6050 là một cảm biến chuyển động tích hợp bộ đo gia tốc ba trục và con quay hồi chuyển ba trục. Hai nhóm cảm biến này cung cấp các phép đo theo ba trục vuông góc, cho phép một hệ thống nhúng quan sát sự thay đổi chuyển động và tốc độ quay của đối tượng gắn cảm biến [1].

Bài lab này tập trung vào cách sử dụng MPU6050 thông qua Device API của EduFramework thay vì thao tác trực tiếp với các thanh ghi của cảm biến. Chương trình khởi tạo module bằng cấu hình mặc định, đọc đồng thời dữ liệu gia tốc, vận tốc góc và nhiệt độ cảm biến, sau đó hiển thị kết quả trên Serial Monitor. Phần mở rộng giới thiệu các API đọc chọn lọc, hiệu chuẩn và giao diện nâng cao cho phép thay đổi địa chỉ I2C cũng như dải đo của gia tốc kế và con quay hồi chuyển [6].

1.2 Mục tiêu

MỤC TIÊU

Các mục tiêu của bài lab bao gồm:

1. Mô tả vai trò của gia tốc kế và con quay hồi chuyển trong một cảm biến chuyển động sáu trục.
2. Giải thích các thành phần cơ bản của giao tiếp I2C gồm SDA, SCL và địa chỉ thiết bị.
3. Sử dụng cấu trúc MPU_Data_t để truy cập dữ liệu gia tốc, vận tốc góc và nhiệt độ cảm biến do EduFramework cung cấp.
4. Sử dụng MPU_Begin() và MPU_ReadData() để xây dựng ứng dụng giám sát dữ liệu MPU6050 trên Serial Monitor.
5. Nhận biết các khả năng cấu hình nâng cao của MPU6050 Device API, bao gồm hiệu chuẩn, thay đổi địa chỉ I2C và lựa chọn dải đo.

2 Kiến thức nền

2.1 gia tốc kế và con quay hồi chuyển

Một IMU điển hình sử dụng ba gia tốc kế và ba rate con quay hồi chuyển được bố trí trên ba trục vuông góc. Con quay hồi chuyển cung cấp các thành phần vận tốc góc của vật thể theo các trục cảm biến, trong khi gia tốc kế cung cấp phép đo specific force theo các trục tương ứng [1].

Trong EduFramework, các giá trị gia tốc kế của MPU6050 được quy đổi sang đơn vị g, còn dữ liệu con quay hồi chuyển được quy đổi sang dps (degree per second). Khi module đứng yên, dữ liệu gia tốc kế vẫn phản ánh ảnh hưởng của trọng lực nên một trục có thể có độ lớn gần 1 g tùy theo hướng đặt cảm biến. Dữ liệu con quay hồi chuyển khi đứng yên thường nằm gần 0 dps, nhưng có sai lệch offset và nhiễu [1] [6].

Ghi chú

con quay hồi chuyển cung cấp vận tốc góc, không trực tiếp cung cấp góc quay. Việc suy ra orientation, pitch, roll hoặc yaw từ dữ liệu IMU cần các bước xử lý bổ sung và không nằm trong phạm vi của bài lab này.

2.2 Giao tiếp I2C

I2C là một bus nối tiếp đồng bộ hai dây. SCL mang tín hiệu clock, còn SDA được sử dụng để truyền dữ liệu. Nhiều thiết bị có thể chia sẻ cùng bus và được phân biệt bằng địa chỉ của từng peripheral [2].

MPU6050 sử dụng I2C làm giao tiếp host trong cấu hình của bài lab. Bit địa chỉ thấp nhất được xác định bởi chân AD0: khi AD0 ở mức thấp, địa chỉ slave là 0x68; khi AD0 ở mức cao, địa chỉ trở thành 0x69 [3].

Luồng dữ liệu của bài lab có thể mô tả bằng sơ đồ:

MPU6050 → I2C → Wire / I2C → MPU6050 Device API → Application

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Bài thực hành sử dụng MaaZEDU Development Board và một module MPU6050. MaaZEDU là board phát triển dựa trên vi điều khiển S32K144 [5]. Module được cấp nguồn 3.3V, dùng chung GND với board và trao đổi dữ liệu qua giao tiếp I2C [3].

Phần cứng	Mô tả
MaaZEDU Development Board	Board phát triển sử dụng vi điều khiển S32K144.
MPU6050 module	Module cảm biến tích hợp gia tốc kế ba trục và con quay hồi chuyển ba trục.
Jumper wires	Kết nối nguồn và các tín hiệu I2C giữa MPU6050 với MaaZEDU.
USB Cable	Kết nối board với máy tính để cấp nguồn, nạp chương trình và sử dụng Serial Monitor.

Bảng 11.3.1. Phần cứng sử dụng trong bài thực hành

3.2 Ánh xạ tín hiệu I2C

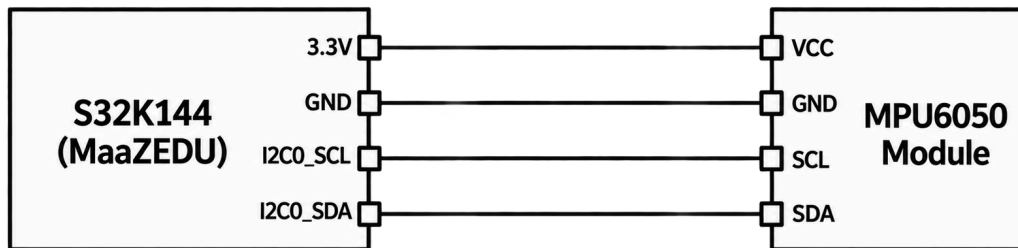
Trong cấu hình EduFramework hiện tại, giao tiếp I2C0 sử dụng các tín hiệu I2C0_SCL và I2C0_SDA. Hai tín hiệu này được sử dụng bởi Wire API và bởi MPU6050 Device API trong bài thực hành [6].

Thành phần	Logical Pin	MCU Pin	Chức năng
MPU6050 SCL	I2C0_SCL	PTA3	I2C Clock
MPU6050 SDA	I2C0_SDA	PTA2	I2C Data

Bảng 11.3.2. Ánh xạ tín hiệu MPU6050 sử dụng trong bài thực hành

3.3 Kết nối MPU6050

Trong cấu hình mặc định của bài lab, VCC được nối với 3.3V, GND nối với GND, SCL nối với I2C0_SCL và SDA nối với I2C0_SDA. Chân AD0 được đặt ở mức thấp bằng cách nối với GND, vì vậy Device sử dụng địa chỉ mặc định 0x68 [3] [6].



Hình 11.3.1. Sơ đồ kết nối MPU6050 với MaaZEDU Development Board

Ghi chú

Chân `INT` không được sử dụng trong bài thực hành này. Dữ liệu được đọc chủ động bằng `MPU_ReadData()` theo chu kỳ của chương trình.

4 EduFramework API

MPU6050 Device của EduFramework cung cấp một nhóm API đơn giản cho các ứng dụng cơ bản và một nhóm API nâng cao cho các trường hợp cần cấu hình chi tiết. Bài thực hành chính sử dụng `MPU_Data_t`, `MPU_Begin()` và `MPU_ReadData()`; các API cấu hình nâng cao được trình bày trong phần Mở rộng [6].

4.1 Cấu trúc `MPU_Data_t`

`MPU_Data_t` gom các giá trị của một lần đọc cảm biến vào một cấu trúc duy nhất. Ứng dụng có thể truy cập trực tiếp từng trường sau khi `MPU_ReadData()` hoàn tất thành công [6].

```
typedef struct
{
    float accelX;
    float accelY;
    float accelZ;
    float gyroX;
    float gyroY;
    float gyroZ;
    float temperature;
} MPU_Data_t;
```

Mã nguồn 11.4.1. Cấu trúc dữ liệu `MPU_Data_t`

Trường	Đại lượng	Đơn vị
accelX	Gia tốc / specific force theo trục X	g
accelY	Gia tốc / specific force theo trục Y	g
accelZ	Gia tốc / specific force theo trục Z	g
gyroX	Vận tốc góc theo trục X	dps
gyroY	Vận tốc góc theo trục Y	dps
gyroZ	Vận tốc góc theo trục Z	dps
temperature	Nhiệt độ cảm biến bên trong MPU6050	°C

Ghi chú

Trường `temperature` phản ánh nhiệt độ cảm biến bên trong MPU6050. Giá trị này không nên được sử dụng như phép đo trực tiếp nhiệt độ môi trường xung quanh [3].

4.2 MPU_Begin()

`MPU_Begin()` khởi tạo MPU6050 bằng cấu hình mặc định của EduFramework. API tự khởi tạo tầng Wire, sử dụng địa chỉ `0x68`, đánh thức cảm biến và thiết lập dải đo gia tốc kể ± 2 g cùng dải đo con quay hồi chuyển ± 250 dps [6].

MPU_Begin

Khởi tạo MPU6050 bằng cấu hình mặc định của EduFramework.

Cú pháp

```
MPU_Begin();
```

Giá trị trả về

`true` khi khởi tạo và kiểm tra giao tiếp thành công; `false` khi không thể giao tiếp hoặc cấu hình cảm biến.

Ví dụ:

```
if (true == MPU_Begin())
{
    /* MPU6050 is ready. */
}
```

4.3 MPU_ReadData()

`MPU_ReadData()` thực hiện một lần đọc measurement frame của MPU6050 và cập nhật đồng thời các trường acceleration, con quay hồi chuyển và temperature trong `MPU_Data_t`. API chỉ trả dữ liệu khi Device đã được khởi tạo thành công [6].

MPU_ReadData

Đọc một measurement frame và cập nhật cấu trúc dữ liệu được truyền vào.

Cú pháp

```
MPU_ReadData(data);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
<code>data</code>	MPU data structure	Nơi lưu dữ liệu acceleration, con quay hồi chuyển và temperature sau khi đọc.

Giá trị trả về

`true` khi dữ liệu được đọc thành công; `false` khi Device chưa được khởi tạo, tham số không hợp lệ hoặc giao tiếp I2C thất bại.

Ví dụ:

```
MPU_Data_t data;

if (true == MPU_ReadData(&data))
{
    Serial1_printFloat(data.accelX);
}
```

5 Bài thực hành

5.1 Yêu cầu

Xây dựng chương trình giám sát dữ liệu chuyển động từ MPU6050 và hiển thị kết quả trên Serial Monitor. Chương trình sử dụng `MPU_Begin()` để khởi tạo Device, sau đó gọi `MPU_ReadData()` sau mỗi khoảng `1000 ms` để lấy acceleration ba trục, con quay hồi chuyển ba trục và nhiệt độ cảm biến.

Serial Monitor sử dụng baud rate `9600`. Khi quá trình khởi tạo thất bại, chương trình phải thông báo lỗi và không tiếp tục đọc cảm biến. Khi một lần đọc thất bại, chương trình in thông báo lỗi cho lần đọc đó và tiếp tục thử lại ở chu kỳ sau.

5.2 Chương trình

Trong `src/main.c`, triển khai chương trình đã được kiểm chứng trên phần cứng như sau:

```
#include "Arduino.h"
#include "MPU6050.h"
int main(void)
{
    MPU_Data_t data;
    setup();
    Serial1_begin(9600U);
    Serial1_println("MPU6050 Sensor Monitor");
    Serial1_println("-----");
    if (false == MPU_Begin())
    {
        Serial1_println("MPU6050 initialization failed.");
        while (1)
        {
            delay(1000U);
        }
    }
    Serial1_println("MPU6050 initialized.");
    while (1)
    {
        if (true == MPU_ReadData(&data))
        {
            Serial1_print("Accel X: ");
            Serial1_printFloat(data.accelX);
            Serial1_print(" g | Y: ");
            Serial1_printFloat(data.accelY);
            Serial1_print(" g | Z: ");
            Serial1_printFloat(data.accelZ);
            Serial1_println(" g");
            Serial1_print("Gyro X: ");
            Serial1_printFloat(data.gyroX);
            Serial1_print(" dps | Y: ");
            Serial1_printFloat(data.gyroY);
            Serial1_print(" dps | Z: ");
            Serial1_printFloat(data.gyroZ);
            Serial1_println(" dps");
            Serial1_print("Temperature: ");
            Serial1_printFloat(data.temperature);
            Serial1_println(" C");
            Serial1_println("-----");
        }
        else
        {
            Serial1_println("MPU6050 read failed.");
        }
        delay(1000U);
    }
}
```

Mã nguồn 11.5.1. Chương trình giám sát dữ liệu MPU6050

5.3 Giải thích chương trình

`setup()` khởi tạo các thành phần nền tảng của EduFramework và `Serial1_begin(9600U)` chuẩn bị kênh Serial Monitor. Sau đó, `MPU_Begin()` thực hiện toàn bộ quá trình khởi tạo cần thiết cho cấu hình mặc định

của MPU6050. Nếu API trả về `false`, chương trình dừng tại vòng lặp lỗi để tránh tiếp tục xử lý với một Device chưa sẵn sàng [6].

Trong vòng lặp chính, `MPU_ReadData(&data)` đọc một measurement frame. Khi phép đọc thành công, toàn bộ dữ liệu đã được chuyển đổi và lưu trong `data`, vì vậy ứng dụng chỉ cần truy cập các trường `accelX`, `accelY`, `accelZ`, `gyroX`, `gyroY`, `gyroZ` và `temperature`. Kết quả được gửi tới Serial Monitor bằng các API Serial đã sử dụng ở các lab trước [6].

Luồng xử lý của ứng dụng có thể tóm tắt:

Chuyển động → MPU6050 → I2C → `MPU_ReadData()` → `MPU_Data_t` → `Serial1` → Serial Monitor
`delay(1000U)` tạo khoảng cách một giây giữa hai lần cập nhật, giúp kết quả trên Serial Monitor dễ quan sát.

5.4 Kiểm chứng

Build và nạp chương trình xuống MaaZEDU Development Board, sau đó mở Serial Monitor với baud rate 9600. Trước tiên, đặt module yên trên một bề mặt ổn định. Khi một trục cảm biến gần thẳng theo phương trọng lực, thành phần acceleration trên trục đó có độ lớn gần 1 g, trong khi gyro trên ba trục nằm gần 0 dps. Các giá trị thực tế có thể lệch nhẹ do hướng đặt module, offset và nhiễu cảm biến [1].

Tiếp theo, nghiêng module để quan sát sự phân bố của acceleration giữa các trục, sau đó xoay module để quan sát sự thay đổi của con quay hồi chuyển. Không yêu cầu các giá trị phải trùng với một bộ số cố định; mục tiêu là kiểm chứng dữ liệu phản ứng hợp lý theo chuyển động thực tế.

Dữ liệu đã quan sát trong quá trình kiểm thử phần cứng có dạng:

```
MPU6050 Sensor Monitor
-----
MPU6050 initialized.
Accel X: 0.060 g | Y: 0.005 g | Z: 1.030 g
Gyro X: 1.901 dps | Y: -1.534 dps | Z: -0.191 dps
Temperature: 46.412 C
-----
```

KẾT QUẢ MONG ĐỢI

Serial Monitor hiển thị liên tục acceleration và con quay hồi chuyển theo ba trục cùng nhiệt độ cảm biến. Khi module được nghiêng hoặc xoay, các giá trị tương ứng thay đổi theo chuyển động; khi module đứng yên, gyro nằm gần 0 dps và acceleration phản ánh thành phần của trọng lực theo hướng đặt cảm biến.

6 Mở rộng

Bài thực hành chính sử dụng Beginner API để giữ cấu hình và luồng chương trình đơn giản. MPU6050 Device còn cung cấp các API đọc chọn lọc, hiệu chuẩn và một Advanced API theo context để ứng dụng có thể chủ động lựa chọn địa chỉ I2C, dải đo và số mẫu hiệu chuẩn khi cần [6].

6.1 Đọc chọn lọc và kiểm tra trạng thái

Ngoài `MPU_ReadData()`, Beginner API cung cấp các hàm đọc riêng từng nhóm dữ liệu:

API	Chức năng
<code>MPU_IsInitialized()</code>	Kiểm tra trạng thái khởi tạo của default Device.
<code>MPU_ReadAcceleration()</code>	Đọc acceleration theo ba trục X, Y, Z.
<code>MPU_Readcon quay hồi chuyển()</code>	Đọc con quay hồi chuyển theo ba trục X, Y, Z.
<code>MPU_ReadTemperature()</code>	Đọc nhiệt độ cảm biến bên trong MPU6050.

Mỗi API đọc chọn lọc thực hiện một lần đọc mới từ cảm biến trước khi trả kết quả. Vì vậy, khi ứng dụng cần đồng thời acceleration, con quay hồi chuyển và temperature, `MPU_ReadData()` phù hợp hơn vì các giá trị được lấy từ cùng một measurement frame và chỉ cần một chu kỳ đọc Device [6].

Ví dụ khi chỉ cần acceleration:

```
float x = 0.0F;
float y = 0.0F;
float z = 0.0F;

if (true == MPU_ReadAcceleration(&x, &y, &z))
{
    /* Use acceleration data. */
}
```

6.2 Hiệu chuẩn với `MPU_Calibrate()`

`MPU_Calibrate()` thực hiện hiệu chuẩn default MPU6050 bằng cách lấy nhiều mẫu khi cảm biến đứng yên và tính các offset cho gia tốc kế và con quay hồi chuyển. Trong implementation hiện tại, Beginner API sử dụng 500 mẫu. Thuật toán giả định module được giữ ổn định với trục +Z gần +1 g trong quá trình hiệu chuẩn [6].

`MPU_Calibrate`

Hiệu chuẩn default MPU6050 và cập nhật các offset được áp dụng cho các lần đọc tiếp theo.

Cú pháp

```
MPU_Calibrate();
```

Giá trị trả về

`true` khi toàn bộ quá trình hiệu chuẩn hoàn tất thành công; `false` khi Device chưa được khởi tạo hoặc có lỗi trong quá trình lấy mẫu.

Ghi chú

Giữ module đứng yên và gần nằm ngang trong suốt quá trình hiệu chuẩn. Di chuyển cảm biến trong khi lấy mẫu sẽ làm các offset thu được không đại diện cho trạng thái tĩnh mong muốn.

6.3 Địa chỉ I2C và chân AD0

Cấu hình mặc định của bài lab nối AD0 xuống GND, tương ứng với địa chỉ 0x68. MPU6050 cũng cho phép đưa AD0 lên mức logic cao để sử dụng địa chỉ 0x69. Cơ chế này cho phép hai MPU6050 cùng tồn tại trên một bus I2C nếu một Device sử dụng địa chỉ 0x68 và Device còn lại sử dụng 0x69 [3].

EduFramework định nghĩa hai hằng địa chỉ:

```
MPU6050_ADDRESS_DEFAULT /* 0x68 */
MPU6050_ADDRESS_ALT     /* 0x69 */
```

Beginner API luôn sử dụng địa chỉ mặc định. Khi cần lựa chọn địa chỉ, ứng dụng sử dụng Advanced API và một context MPU6050_t riêng. Trong Advanced API, tầng Wire phải được khởi tạo trước khi gọi MPU6050_begin() [6].

Ví dụ nguyên tắc:

```
MPU6050_t mpu;
Wire_begin();
if (true == MPU6050_begin(&mpu, MPU6050_ADDRESS_ALT))
{
    /* Device at address 0x69 is ready. */
}
```

Mã nguồn 11.6.1. Khởi tạo MPU6050 bằng địa chỉ I2C tùy chọn

6.4 Tùy chỉnh dải đo

Product Specification của MPU6050 hỗ trợ bốn dải full-scale cho gia tốc kế và bốn dải full-scale cho con quay hồi chuyển. EduFramework expose các lựa chọn này thông qua Advanced API [3] [6].

gia tốc kế	con quay hồi chuyển
MPU6050_RANGE_2_G	MPU6050_RANGE_250_DEG
MPU6050_RANGE_4_G	MPU6050_RANGE_500_DEG
MPU6050_RANGE_8_G	MPU6050_RANGE_1000_DEG
MPU6050_RANGE_16_G	MPU6050_RANGE_2000_DEG

Hai API cấu hình tương ứng:

```
MPU6050_setgia tốc kếRange(&mpu, MPU6050_RANGE_4_G);
MPU6050_setGyroRange(&mpu, MPU6050_RANGE_500_DEG);
```

Dải đo lớn hơn cho phép cảm biến biểu diễn chuyển động có biên độ lớn hơn, trong khi độ nhạy theo số đếm trên mỗi đơn vị đo thay đổi theo dải đã chọn. Việc lựa chọn dải đo nên dựa trên biên độ chuyển động của ứng dụng thay vì luôn sử dụng dải lớn nhất [3].

6.5 Advanced API theo context

Advanced API lưu địa chỉ, dải đo, dữ liệu đo và các offset hiệu chuẩn trong MPU6050_t. Mô hình này cho phép application kiểm soát cấu hình Device thay vì phụ thuộc vào context mặc định của Beginner API [6].

Các API chính gồm:

Advanced API	Vai trò
<code>MPU6050_begin()</code>	Khởi tạo một context với địa chỉ I2C được lựa chọn.
<code>MPU6050_setgia tốc kếRange()</code>	Thay đổi dải đo gia tốc kế.
<code>MPU6050_setGyroRange()</code>	Thay đổi dải đo con quay hồi chuyển.
<code>MPU6050_read()</code>	Đọc measurement frame vào context.
<code>MPU6050_calibrate()</code>	Hiệu chuẩn với số mẫu do application lựa chọn.
<code>MPU6050_getAccelerationX()</code> , <code>MPU6050_getAccelerationY()</code> , <code>MPU6050_getAccelerationZ()</code>	Đọc acceleration từng trục đã lưu trong context.
<code>MPU6050_getGyroX()</code> , <code>MPU6050_getGyroY()</code> , <code>MPU6050_getGyroZ()</code>	Đọc con quay hồi chuyển từng trục đã lưu trong context.
<code>MPU6050_getTemperature()</code>	Đọc nhiệt độ đã lưu trong context.

Beginner API phù hợp khi chỉ cần một MPU6050 với cấu hình tiêu chuẩn. Advanced API phù hợp khi application cần địa chỉ khác, dải đo khác, số mẫu hiệu chuẩn tùy chọn hoặc cần tự quản lý context Device.

6.6 Bài tập mở rộng

Cấu hình MPU6050 ở địa chỉ `0x69` bằng cách đặt `AD0` ở mức logic cao và sử dụng Advanced API để:

- khởi tạo một `MPU6050_t` riêng;
- lựa chọn gia tốc kế range ± 4 g ;
- lựa chọn con quay hồi chuyển range ± 500 dps ;
- thực hiện hiệu chuẩn với số mẫu do chương trình lựa chọn;
- đọc dữ liệu và hiển thị acceleration cùng con quay hồi chuyển trên Serial Monitor.

Không sử dụng `MPU_Begin()` trong bài tập này. Mục tiêu là thực hành trực tiếp với `MPU6050_begin()` , các API cấu hình range và `MPU6050_calibrate()` .

KẾT QUẢ MONG ĐỢI

MPU6050 được khởi tạo thành công tại địa chỉ `0x69` , dữ liệu tiếp tục được đọc và hiển thị sau khi thay đổi dải đo, và các giá trị gyro ở trạng thái đứng yên được cải thiện sau quá trình hiệu chuẩn phù hợp.

7 Tài liệu tham khảo

- [1] University of California, Berkeley, *Strapdown inertial navigation* , Rotations.
- [2] Cornell University, *Inter-Integrated Circuit (I2C)* , ECE 4760 - Designing with Microcontrollers.
- [3] InvenSense Inc., *MPU-6000 and MPU-6050 Product Specification* , PS-MPU-6000A-00, Rev. 3.4, 2013.
- [4] NXP Semiconductors, *S32K1xx MCU Family - Data Sheet* , S32K1XX, Rev. 15, 2026.
- [5] MaaZEDU, *MaaZEDU Development Board Guide* , Rev. 1.0, 2025.
- [6] EduFramework, *MPU6050 Device API* , EduFramework Source Code.